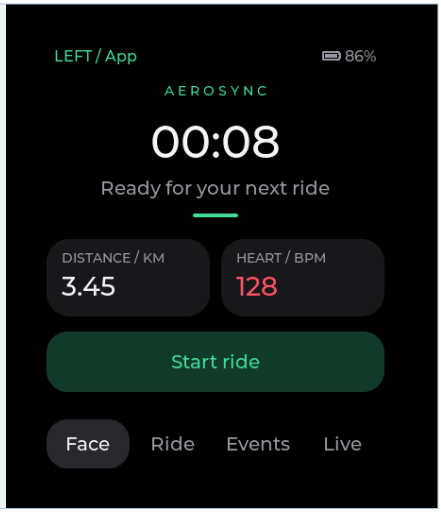
AeroSync Watch Firmware

System, feature and companion-app connectivity report

Conference technical brief · source-audited implementation · 21 October 2026

Abstract
AeroSync is embedded firmware for a Waveshare ESP32-S3-Touch-AMOLED-2.06 wearable. It combines a four-page touch interface, 6-axis inertial sensing, local CSV logging, and a custom bidirectional Bluetooth Low Energy (BLE) protocol for a Ride Sensor Capture companion app. The central design decision is graceful autonomy: the watch streams 50 Hz motion data when a phone is present, but can begin and preserve a labelled ride locally when the companion is absent or stale.

Evidence basis. All implementation claims were traced to current source, headers, build configuration, partition map and simulator tests. The legacy README was used only where it agrees with executable code.



Research contribution and scope

- Integrated sensing node: QMI8658 sampled at 50 Hz, displayed locally, recorded as labelled CSV and streamed as SI-unit BLE packets.
- Resilient interaction: phone-confirmed control when fresh; automatic local fallback when absent or stale; explicit ride-end confirmation.
- Dual-watch identity: persistent Left/Right role changes advertising name and telemetry role for concurrent body-placement datasets.
- Reproducible evidence: current simulator produced all figures and passed scripted state/interaction assertions.

Audit item	Observed implementation
Target	ESP32-S3R8; 410×502 CO5300 AMOLED; FT3168 touch; QMI8658 IMU
Firmware stack	Arduino/PlatformIO; C++17; LVGL 9.2.2; Arduino_GFX 1.6.0; FreeRTOS
Primary modes	Companion-controlled ride; standalone ride; live signal monitor
Outputs	50 Hz BLE telemetry; subject/session CSV on microSD or LittleFS
Verification	Simulator test PASS; embedded build recorded in manifest

Figure 1. Current Face page (simulated connected state); direct simulator output, not a design mock-up.

1. Hardware and firmware architecture

The boot order is persistent role → mutex-protected I²C → display/touch → IMU → storage → recorder → application sync → BLE → UI. This avoids reset and shared-bus races during initialization.

Subsystem	Configuration / implemented behavior
Compute	ESP32-S3 dual-core; 16 MB addressable flash; 8 MB octal PSRAM declared. UI on core 1; sensing, BLE publication and storage on core 0.
Display + touch	CO5300 410×502 QSPI AMOLED with 22 px column offset; FT3168 touch on 400 kHz shared I²C; two 410×48×2-byte internal-RAM render buffers when available.
Motion sensor	QMI8658 probe 0x6B/0x6A then 0x68–0x6F; ±8 g, ±512 deg/s, 1 kHz ODR; one coherent 12-byte six-axis burst.
Storage	1-bit SDMMC microSD preferred; LittleFS fallback. Flash map includes 6 MB app and 9.875 MB data partition.
Power state	AXP2101 percentage/charging first; GPIO10 ADC ×2 fallback. GATT battery every 5 s; companion report every 30 s.
Persistence	Left/Right role, face and brightness are held in NVS Preferences.

Concurrent data path

Producer / consumer	Cadence and isolation	Purpose
esp_timer → sampler	20,000 µs; priority 5	Read/timestamp IMU; count overruns; update live values
sampler → BLE queue	Non-blocking; 8; drop oldest	Maintain freshness without latency accumulation
sampler → sample queue	Non-blocking; 512 samples	Isolate timing from storage stalls
sd_writer	Priority 3; 4 KB; 2 s flush	Format CSV, close durably, append session index
loop / LVGL	10 Hz presentation refresh	Touch, state and command dispatch

Board interfaces and fixed pin map

Interface	Pins / address	Notes
AMOLED QSPI	CS12, SCK11, D0–D3=4–7, RST8	TE13 defined but unused
Shared I²C	SDA15, SCL14; 400 kHz	Recursive mutex; 50 ms timeout
Touch	FT3168 0x38; INT38, RST9	LVGL pointer scan every 16 ms
IMU	QMI8658 0x6B/0x6A; INT21	INT unused; esp_timer cadence
microSD	CLK2, CMD1, D0=3; 1-bit	No card-detect input used

Signal semantics

- Local CSV uses g and degrees/s; BLE converts to m/s² and rad/s.
- Live view exposes X/Y/Z acceleration, resultant magnitude, gyroscope, backend and samples written.
- Read failures keep the last display value and increment errors; failed frames are not recorded.

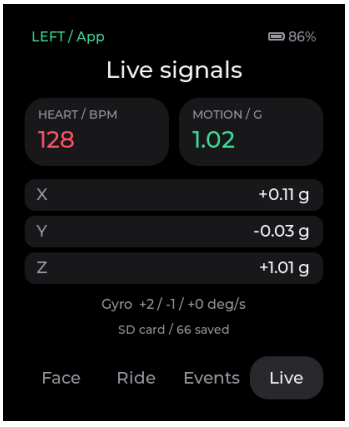


Figure 2. Live sensing page: local inertial observability and storage-write count.

2. Complete user-visible feature catalogue

Page / interaction	Features and state-dependent behavior
Face	Digital clock and Ride Focus speed face; long-press toggles. Time requires phone epoch/time-zone sync. Distance, heart rate and ride state are companion-fed.
Ride	GPS speed/availability, duration, distance, sync state and Start/End. End opens Save & end / Keep riding confirmation.
Events	Bump=1, Turn=2, Speed=3, Finish=4. Disabled unless ride active; standalone Finish returns sample label to Normal/0.
Live	Heart-rate mirror plus local acceleration, magnitude and gyro; sampling continues independently of disk recording.
Navigation	Bottom tabs + cyclic swipe; 26 px bezel-safe inset; 100 ms refresh throttle.
Role setup	Left/Right overlay; role locked while recording; role change restarts advertising.
Appearance	Phone-controlled face/brightness; watch reports settings on connect/ping; local long-press face toggle.
Feedback	App/Local/Standalone, pending/saving, storage, stale GPS, missing sensor and retry guidance.

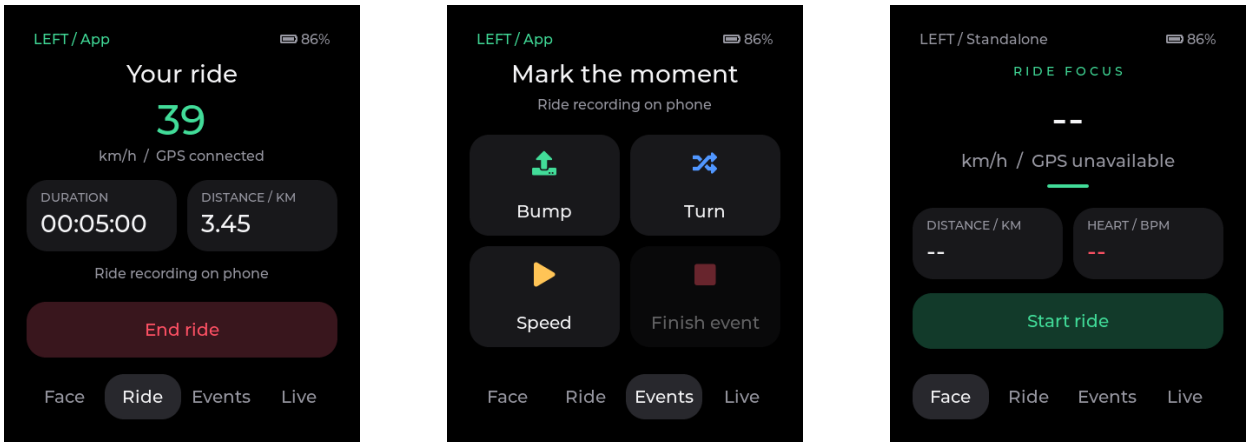


Figure 3. Executable UI evidence: Ride metrics, event labelling and phone-disconnected local mode.

Ride-control state model

Connected + recently responsive phone → watch sends request ID → UI blocks duplicates → app confirms. No phone or stale handshake → watch enters Local mode → recorder starts/stops immediately → later phone snapshots cannot cancel the local ride. Missing confirmation after 12 s clears pending state and requests phone-side review.

Human factors: 64–116 px targets, bottom navigation, visible modes, destructive confirmation and disabled impossible actions suit brief motion-constrained use. No haptic or audio feedback is implemented.

3. Companion-app connectivity and wire protocol

The watch is a BLE GATT server advertising ESP32-Watch-LEFT/RIGHT/UNSET; manufacturer data is FF 02 + role. Advertising exposes FFE0 and Battery Service. Requested interval is 15–30 ms; requested MTU is 512.

GATT endpoint	Properties	Direction and payload
FFE1 Telemetry	Read, Notify	Watch → app: packed 32-byte IMU at nominal 50 Hz
FFE2 Command RX	Write, Write NR, Read	App → watch: ≤64-byte opcode messages, queued for UI task
FFE3 Control TX	Read, Notify, Indicate	Watch → app: requests/state/settings/battery; subscription required
180F / 2A19	Read, Notify	Standard one-byte watch battery percentage

App → watch commands on FFE2

Op	Meaning	Decoded fields / effect
01	Trip	0 stop / 1 start; optional name; starts local backup
02	Event	u16 BE class + optional name; updates per-row label
03	Telemetry	speed×10 u16 BE; distance m u32 BE; HR; GPS lock/satellites
04/05	Time / battery	epoch u32 BE + timezone i16; percent + charging
06/0C	Ping / Hello	refresh readiness; return state/battery/settings
07/08	Settings / elapsed	face + brightness; elapsed seconds u32 BE
09	Result	request ID + success/failure
0A/0B	Role / control	select role; restore phone control only while idle

Watch → app commands on FFE3

Op	Meaning / content
10/11	Trip toggle or event trigger; request ID appended
12/13/14	Export/finalize; watch battery/charging/mV; keep-alive
15	Face + brightness
16	Protocol v2, role, local/trip flags, recorder, SD, written count

32-byte telemetry packet

timestamp_ms u32 | sequence u16 | role u8 | accel_x/y/z float32 m/s² | gyro_x/y/z float32 rad/s | battery u8. Queue is non-blocking and eight entries deep; oldest is discarded on overflow. Sequence gaps expose loss.

App requirements: subscribe to FFE1/FFE3; parse packed little-endian ESP32 telemetry; send big-endian command integers where specified; echo request IDs via 09; refresh telemetry within 6 s; transmit hello/command within 8 s for companion-ready state.

Telemetry byte layout and connection lifecycle

Offset	Width	Field
0	4	device-local timestamp_ms (uint32)
4	2	monotonic sequence (uint16; wraps at 65,535)
6	1	role: 1 Left, 2 Right
7	12	accel X/Y/Z float32 in m/s²
19	12	gyro X/Y/Z float32 in rad/s
31	1	battery percentage; 255 may mean unknown

Connect resets queues and sends battery/appearance; first phone message establishes phoneSeen. Disconnect clears pending and phone-derived metrics, restarts advertising and leaves inertial sampling active. Values older than 6 s are hidden; companion-ready also requires a phone message within 8 s.

4. Recording, resilience, verification and limitations

Local research dataset

Persistent roles map to LEFT_Wrist or RIGHT_Wrist. A ride uses activity ‘Ride - row labels’, initially class 0; events update each subsequent sample label. data.csv is continuous per subject and sessions.csv preserves block boundaries and the next time base.

File / mechanism	Implemented content
data.csv	timestamp_ms, acc_x/y/z, gyro_x/y/z, label, session; valid 50 Hz frames
sessions.csv	session, class, activity_name, start_ms, end_ms, samples, sample_rate_hz
Storage choice	microSD preferred, LittleFS fallback; refuse missing IMU/storage, <256 KiB, directory/open errors
Durability	4 KB staging; ≤2 s flush; queue drain + close before index; saturation/overrun counters
Export engine	Flash→SD rebasing merge exists; current ride UI exposes neither export nor periodic card-remount

Verification evidence

Evidence	Result and interpretation
Source audit	16 headers, 13 C++ modules, build config, partition map, simulator and sample data scanned; source overrides stale README.
Simulator	PASS: navigation/bounds, confirmation, duplicate start, event reset, disconnect/local, stale telemetry, 12 s timeout.
Screenshots	Figures regenerated from current 410×502 LVGL simulator on 21 Oct 2026.
Build	PlatformIO target invoked; final result and image size are in the accompanying manifest.

Failure containment implemented in code

Condition	Contained behavior
BLE congestion	Drop oldest telemetry, retain newest, increment drop counter
No microSD at boot	Use LittleFS without blocking ride capture
Write/flush failure	Close best-effort, publish Error, stop local ride and report
IMU unavailable	Retry every 2 s for live reads; refuse recording until ready
Sample queue full	Count loss; never stall 50 Hz producer
Invalid/duplicate action	Ignore pending duplicate; disable impossible controls

Limitations and research risks

- No pairing, bonding, authentication, encryption policy or message authentication is configured; ride control and telemetry are not security-qualified.
- FFE1 is a packed native struct: explicit little-endian IEEE-754 parsing and protocol versioning are needed. There is no checksum, retransmission or synchronized timestamp.
- QMI8658 CTRL5=0 disables LPF, contradicting a README/config comment claiming ~27 Hz anti-alias filtering. Current code has no sensor LPF.
- Phone GPS/distance/HR/battery are displayed but not stored locally; no on-watch GPS or HR sensor is implemented.
- Software simulation/build do not establish radio range, packet loss, battery life, calibration, physical timing jitter, waterproofing or medical validity.

Conclusion

AeroSync is a credible research prototype for synchronized wearable inertial capture, with responsive interaction, freshness-oriented BLE streaming and local evidence preservation. Before field deployment or instrument-validity claims, add BLE security, versioned serialization, synchronized timestamps, calibration, physical trials and quantitative power/link results.

Primary evidence: src/{main,ui,recorder,ble_manager,app_sync,storage}.cpp; include/{config,ble_manager,app_sync,board_pins}.h; platformio.ini; partitions_16mb.csv; tools/simulator/sim_main.cpp.